

Team 2 - Billy's Amazing Team

Member Names: Mitchel Bekink, Alyx Bruno-Bamford, Ashley Bryan, Rajul Chawla, Bosco Lau, Kacey Van Der Walt, Chris Grzywacz

Testing Methods and Approaches (4a):

The first component of testing involves planning the scope of our tests, and what proportion of the total tests conducted, are of which scope. The main three scopes that can be identified are narrow tests (unit tests), medium tests (integration tests) and large tests (end-to-end tests). As our project is relatively small, our integration tests and end-to-end tests are one in the same, as the adjacent components of our game make up the entire system, so each of our integration tests is in essence an end-to-end test.

With our test scopes identified, we need to ensure that our tests fall into a good testing pattern. For our project we decided to have our tests fit the test pyramid pattern, where 20% of our overall tests would be integration tests and 80% would be unit tests.

The next factor in our testing was how to pick inputs for our tests. We decided against random sampling as while this is quick and easy, it can be prone to missing potential bugs and errors, as well as not providing good test coverage. Instead, we partitioned the input space based on the specific tests, and then chose tests that fit into these different partitions, ensuring that each one had at least one test associated with it. On top of this, we made sure to test all the boundaries between these partitions, which ensured that edge cases were tested alongside the other inputs.

With our inputs decided, we needed to ensure that the tests we ran provided ample coverage of our code. We needed to ensure both structural coverage and statement coverage, and to do this we used two automated testing softwares called JUnit and JaCoCo. These provided us with reports on our tests, showing us the coverage as a percentage, which allowed us to hone in our tests to produce as high of a coverage as possible.

With all this established, to save us time, as well as to ensure that each new build of our game was thoroughly tested, we used GitHub actions to automatically run our tests on every push to the main repository. This enabled quick, and frequent testing, to ensure that no bugs slipped under the radar when we were merging branches.

Tests that were not automatable, or able to be carried out within the code were carried out by the team and each of these qualitative tests was documented specifically, to ensure they were as traceable and replicable as our written tests.

We ensured that every requirement had at least one test written for it, either an automated test or a qualitative test, and we displayed these results in a testing table. This, combined with all of the above mentioned techniques and approaches, ensured that our game was as error-free as possible, ensuring a smooth experience for player, and future developers.

Test Report (4b)

Below are tables that link our tests to our requirements. Automated tests were used whenever possible and these have been listed, however when manual tests were required we have provided the reason as to why they were chosen, as well as listing a test ID so that more details can be looked up in [the Manual Testing Report](#) (note that all NFRs are tested manually due to them being untestable through code). The table is shown below:

ID	Test Type	Test Location or Description
UR_UX	Manual - ALL TESTS PASSED	This is qualitative so an automated test would not be possible. Positive feedback from the User Evaluation will test our UX layout.
UR_BUILDINGS	Automated - ALL TESTS PASSED	BuildingSlotTest.java, testBuildingPlacement()
UR_UI	Manual - ALL TESTS PASSED	This is qualitative so an automated test would not be possible. Positive feedback from the User Evaluation will test our UX layout.
UR_DISPLAY_TIMER	Manual - ALL TESTS PASSED	As this requirement is about something being visible, an automated test is impossible, see manual test T1 in the test report linked above.
UR_BUILDING_COUNTER	Manual - ALL TESTS PASSED	This was done manually due to the fact that automatic testing would require a significant code refactor, see manual test T2.
UR_PAUSE	Both - ALL TESTS PASSED	TimerTest testTimerPause() tests that pause functionality works. To test that the user is always able to pause, see manual test T3.
UR_UPGRADE	N/A	This requirement was not implemented
UR_SATISFACTION_METER	Both - ALL TESTS PASSED	SatisfactionBarTest.java, testSingleThought() testMultiThought(). The automated test checks that satisfaction was calculated correctly. To check that this is displayed correctly see manual test T4.
UR_ACCESSABILITY_SETTINGS	N/A	This requirement was not implemented
UR_EVENTS	Automated - ALL TESTS PASSED	EventTest.java, testEventCreation() testEventTiming()
UR_KEYBINDS	N/A	This requirement was not implemented
UR_SCOREBOARD	Both - ALL TESTS PASSED	We tested adding to the scoreboard automatically: ScoreTest.java, testScore(). LeaderboardTest.java testLeaderboardOrdering() testLeaderboardLimit(). We manually tested if it was shown and saved between games, due to automation complexity. See manual test T5.
UR_ACHIEVEMENTS	Automated - ALL TESTS PASSED	AchievementsTest.java, testAchievementManager()
FR_SYS_REPONSE	Manual - ALL TESTS PASSED	This is qualitative so an automated test wouldn't be possible. Feedback from User Evaluation will test our system responsiveness.
FR_BUILDING_PLACEMENT	N/a	Due to the architecture it is impossible for a building to be put outside a building slot.
FR_BUILDING_SATISFACTION_GAIN	N/a	This requirement was not implemented

FR_CONSTRUCTION_SATISFACTION_LOSS	Automated - ALL TESTS PASSED	EventManagerTest.java testConstruction()
FR_TIMER_DISPLAY	Both - ALL TESTS PASSED	TimerTest.java testTimerLength() tests that the timers length is correct. TimerTest.java testTimerDisplay() tests that the format used to display the time is right. The rest is done manually due it being hard to automate. See manual test T6.
FR_BUILDING_COUNTER	Manual - ALL TESTS PASSED	This is tested identically to FR_BUILDING_COUNTER due to the similarity in requirement, see manual test T2
FR_SATISFACTION_BREAK_DOWN	Both - ALL TESTS PASSED	ThoughtTest.java testBasicThought() testServiceThought() tests that thoughts work and affect the score. Displaying them is tested manually see manual test T7
FR_BUILDING_OVERLAP	Automated - ALL TESTS PASSED	BuildingSlotTest.java testBuildingCollision() testBuildingFits()
FR_BUILDING_CONSTRUCTION	Automated - ALL TESTS PASSED	BuildingSlotTest.java, testBuildingPlacement()
FR_SATISFACTION_TRACK	Manual - ALL TESTS PASSED	As this requirement is about something visible, an automated test is impossible, see manual test T8
FR_COLOUR_BLIND_MODE	N/A	This requirement was not implemented
FR_MOUSE_SENSITIVITY_SLIDER	N/A	This requirement was not implemented
FR_RANDOM_EVENT	N/A	This requirement was not implemented, due to taking the game in a different direction
FR_SET_EVENT	Automated - ALL TESTS PASSED	As this is conducted through a thought, it is covered by this test: EventTest.java, testEvent()
FR_SATISFACTION_PAUSE	Manual - ALL TESTS PASSED	As this requirement is about visible elements, a manual test is needed. See manual test T9.
FR_TIMER_PAUSE	Automated - ALL TESTS PASSED	TimerTest.java testTimerPause()
FR_MULTIPLE_BUILDINGS	Automated - ALL TESTS PASSED	EventManagerTest.java testMultiConstruction()
FR_UPGRADE_SATISFACTION_GAIN	N/A	This requirement was not implemented
FR_CALCULATE_SCORE	Manual - ALL TESTS PASSED	This requirement is about something being visible at the end of the game so an automated test is impossible. See manual test T10.
FR_DISPLAY_SCORES	Manual - ALL TESTS PASSED	This requirement is about visible elements and hence an automated test is impossible. See manual test T11.
FR_STORE_SCORE	Automated - ALL TESTS PASSED	LeaderboardTest.java testLeaderboardOrdering() testLeaderboardLimit()
FR_CHECK_ACHIEVEMENTS	Automated - ALL TESTS PASSED	AchievementsTest.java, testAchievementManager()

NFR_SYS_RESPONSE_SPEED	Manual - ALL TESTS PASSED	See manual test T12
NFR_COUNTER_VISIBILITY	Manual - ALL TESTS PASSED	See manual test T13
NFR_TIMER_VISIBILITY	Manual - ALL TESTS PASSED	See manual test T14
NFR_SATISFACTION_MULT	N/A	Not implemented
NFR_SATISFACTION_METER_VISIBILITY	Manual - ALL TESTS PASSED	See manual test T15
NFR_READABILITY	Manual - ALL TESTS PASSED	See manual test T16
NFR_EVENT_COOLDOWN	Manual - ALL TESTS PASSED	See manual test T17
NFR_MULTIPLE_CONSTRUCTION	Manual - ALL TESTS PASSED	See manual test T18

There are other tests that were not mentioned above as they are not directly connected to a requirement, but instead are there to either test our architecture (for example ensureAssets() prevents a regression), or to increase test coverage, each of which will help ensure the completeness and correctness of our code, these other tests are listed here:

- achievementTest: testDoubleAchievement(), testLoadSave()
- assetsTest: ensureAssets()
- buildingSlotTest: testBuildingSlot(), testBuildingPreview()
- buildingTest: testBuilding()
- contentLoaderTest: testLoad(), testSave()
- eventManagerTest: testUpdate(), testOneOfEach(), testBuildingMissing(), testPerfectLevel(), testEventDedup()
- satisfactionBarTest: testColourChange(), testEmpty(), testOutputString()
- timerTest: testEnd()

It can be seen that most of our implemented user and functional requirements are covered by automated testing which is the desirable outcome. Approximately 63% of our implemented requirements are tested at least partially automatically (44% are fully automated, 19% are both and 37% are fully manually tested). On top of the success with our automated vs manual testing, all tests upon final build passed, including both the automated and manual tests. This means that all of our requirements were met, as proven by testing, meaning we can be confident that our game will meet the brief as set out by our customer.

In terms of ensuring our game is error-free and will not crash, we need to look toward our coverage report which was generated by JaCoCo. The results of this showed a total instruction coverage of 53% and a branch coverage of 67%. While these numbers may seem lower than ideal at first, a great deal of these uncovered lines and branches come from our main class, where the majority of code is purely library-based (mainly LibGDX), or methods from other classes. The library code is not code that we have written ourselves, and the methods used from these libraries have been tested vastly, both by the authors as well as other users, hence testing these lines ourselves would purely be a waste of time. The average instruction coverage excluding main was much higher at 96.2%, and the branch coverage excluding main was also much higher at 89.5%. With such high coverage of our code, we can safely say that there is a high probability that our code is error-free.

With the combination of our requirement test report, our coverage report, and the fact that no tests failed upon final build, we can be confident that our code has met the remit in terms of completeness and correctness. Links to The test coverage and JUnit reports can be found in the next section.

Testing Material Links (4c):

- [JaCoCo Coverage Report](#) and [Repository](#) (full link: <https://github.com/ashhhleyyy/eng1-cohort2-group2-part2/tree/main/website/JaCoCo>)
- [JUnit Tests Report](#) and [Repository](#) (full link: <https://github.com/ashhhleyyy/eng1-cohort2-group2-part2/tree/main/website/JUnit>)
- [Manual Testing Report](#) (full link: https://ashhhleyyy.github.io/eng1-cohort2-group2-part2/part_2_software_test_reports/ManualTestsReport.pdf)